Fundamental of C

The C character sets

It is also called alphabet of C

It is the set of characters that are used to from words, numbers and expressions in C.

The C character sets consists of following four groups of characters

1. **Letters** :
   - Uppercase : A, B, …, Z
   - Lowercase: a, b, …z
2. **Digits**: 0, 1, …, 9
3. **White space:** Blank space, Horizontal tab, new line, carriage return, form feed.
4. **Special characters**:

, comma . Period ; semicolon : colon ? question mark

‘ apostrophe “ quotation mark ! exclamation mark | vertical bar

/ slash \ backslash ~ tilde _ underscore $ dollar sign

% percent sign & ampersand ^ caret * asterisk

- minus sign + plus sign < opening angle bracket

> closing angle bracket (left parenthesis) right parenthesis

[left bracket] right bracket { left brace } right brace

number sign

By: Dinesh Khadka

comments

- part of program that are not compiled and executed are comments.
- That they make the program readable and understandable
- C supports two types of comments

Single Line Comment

Line comment means we can give comment for a single line. For example

//C programming is more interesting.

Here // is used as a single line comment, it has no ending symbol.

Any thing in a line followed by // will be commented.

Multiple Line Comment

It means user can give comment for multiple line with between the symbols /* */.

Multiple line comment start with /* and end with */.

Every /* has its corresponding */, otherwise every thing that followed /* will be commented.

For example

/* C programming has rich set of operators*/ or we can write as

```
/* C programming
        has rich
          set of operators*/
```

By: Dinesh Khadka

Tokens

- in a C program the smallest individual units recognized by the C compiler are known as C tokens.
- E.g. of C tokens are: keywords (e.g. float, while), identifiers (e.g. num, sum), constants (e.g. -15.5, 100), string literals (e.g. “ABC”, “year”), operators (e.g. +,-,*,/) and special symbols (e.g. [], {}, (), ,).
- C program are written using these tokens and the syntax of the language.
- Types of tokens in C are
  - Keywords
  - Identifiers
  - Constants
  - String constants
  - Operators
  - Special symbols

By: Dinesh Khadka

Keyword

- Keywords are predefined words in C programming language.
- All keywords have fixed meaning and these meanings cannot be changed.
- Keywords serve as basic building blocks for program statements.
- Keywords are also called reserved words because they are used for pre-defined purposes and cannot be used as identifiers.
- All keywords must be written in lowercase.
- There are 32 keywords which are listed below.

auto	double	int	struct	break	else	long
switch	case	enum	register	typedef	char	extern
return	union	const	Float	short	while	continue
for	signed	void	default	goto	sizeof	volatile
do	if	static	unsigned			

Dinesh Khadka

Identifiers

- Identifier refers to the names of variables, functions, and arrays.
- These are the user's defined names and consist of sequences of letter ,digits, and underscore, with letter/underscore as first characters.
- Both upper and lower case are permitted, although lower case letter are commonly used.
- Rule for identifiers
  - First character must be an alphabet (or underscore)
  - Must consist of only letters, digits, or underscore.
  - Only first 31 characters are significant.
  - Can not use keyword.
  - Must not contain white space

Example of valid Identifiers

a, length, area2, area_box, _sum, TOTAL etc.

Example of invalid Identifiers

for, int, do etc – keyword can not be used as identifiers

my name --- white space can not be used.

"s" -- illegal use of character "

By: Dinesh Khadka

Data Types in C

- Data types define the way in which values and ranges of values are represented in system.
- It defines the set of values that a variable can store.
- C language is rich in its data types. C supports three classes of data types:
  - Fundamental (primitive) data types
  - Derived data types
  - User-defined data types

By: Dinesh Khadka

Fundamental data types

- Also called primary or basic data types
- These are built in data type of C language.
- Also used to build other data types
- There are four types of primary data types
  - Integer data types
  - Floating points types
  - Character data types
  - Void data type

By: Dinesh Khadka

Integer data types

- Integers are the whole numbers with ranges of values supported by particular machine. (i.e. non-fractional numbers)
- It can be signed or unsigned.
- In order to provide some control over the ranges of numbers and storages, C has three classes of integer storages— **integer**, **short integer and long integer**.
  - Integer
    - Represented by keyword **int**
    - **Size –** 2|4Bytes depending on system.
    - Range of value : -32768 to 32767 in 2 Byte system and -2, 147, 483, 648 to -2, 147, 483, 647 in 4 byte system.
    - By default all integer are signed unless explicitly specified.
      - Signed integer
        - Default is signed integer
        - Represent both +ve and –ve integers
        - Keyword for data qualifier is **signed int or int**
        - **Range is:** -32768 to 32767 in 2 Byte system and -2, 147, 483, 648 to 2, 147, 483, 647 in 4 byte system. (one bit is reserved for sign)
        - Conversion character is **d**

By: Dinesh Khadka

- Unsigned integer
  - Have to declare explicitly
  - Represents only +ve value
  - Keyword for data qualifier is **unsigned int**
  - It reserve 2|4 Bytes of memory based on system
  - Range: 0 to 65,535 in 2 bytes and 0 - 4,294, 967, 295 in 32 bit system
  - Conversion character is **u**
- Short integer
  - It is also signed and unsigned (default is signed)
  - Signed short integer
    - +ve and –ve
    - Size: 1| 2 Bytes
    - Value: -128 to 127 in 16 bit system and -32,768 to 32767 in 32 bit system.
    - Keyword: **short int** or **signed short int**
  - Unsigned short integer
    - +ve value between 0 to 255 in 16 bit system and 0 to 65, 535 in 32 bit system.
    - Keyword **unsigned short int**

By: Dinesh Khadka

- Long integer
  - Signed long integer
    - Default and +ve and –ve value
    - Keyword: **long int** or **signed long int**
    - Size : 4 Bytes
    - Value: -2,147,483,648 to 2,147,483,647
  - Unsigned long integer
    - -ve
    - Keyword: **unsigned long int**
    - Size: 4 Bytes
    - Value: 0 to 4,294, 967, 295

By: Dinesh Khadka

Floating point types

Floating data are the real numeric values. C support three types of float data types

a) float type
   - Represent the real numeric values with 6 digits of precisions.
   - Size: 4 Bytes
   - Value: 3.4e-38 t 3.4e +38
   - Keyword: **float**

b) double type
   - Represent the real numeric values with 14 digits of precisions.
   - Size: 8 Bytes
   - Value: 1.7e-308 to 1.7e+308
   - Keyword: **double**

c) long double type
   - Represent the real numeric values with 18digits of precisions.
   - Size: 10 Bytes
   - Value: 3.4e- 4932 to 1,1e+4932
   - Keyword: **long double**

By: Dinesh Khadka

Character data types

- A single character can be defined as character type data.
- Characters are usually stored in 1 bytes of internal storage.
- The qualifier signed and unsigned may be explicitly used to char.
- Range of unsigned char have 0 to255, and signed have -128 to 127. (as character are stored as integer not character for example 'A' is represented by 65)
- Keyword : **char**

Void data types

- The void type has no value.
- Keyword: **void**
- This is usually used to specify the return type of function that return no value to the calling function.

By: Dinesh Khadka

2. Derived data types

Derived data types are also known as secondary data types.

These are the collection of same or different primary data types.

The derived data types are

- Array
- Structure
- Union
- Pointer etc

By: Dinesh Khadka

User Defined Data Types

- C language supports a feature where user can define an identifier that characterizes an existing data type.
- This user defined data type identifier can later be used to declare variables.
- In short its purpose is to redefine the name of an existing data type.
- Syntax:

  typedef <type> <identifier>
- Example: **typedef int number**;
- We have just changed the default keyword "**int**" to declare integer variable to "**number**".
- Another user defined data type is enumerated data type.
- It is defined as follows:

  enum identifier {value1, value2, value3, ……, valuen};

  Example:

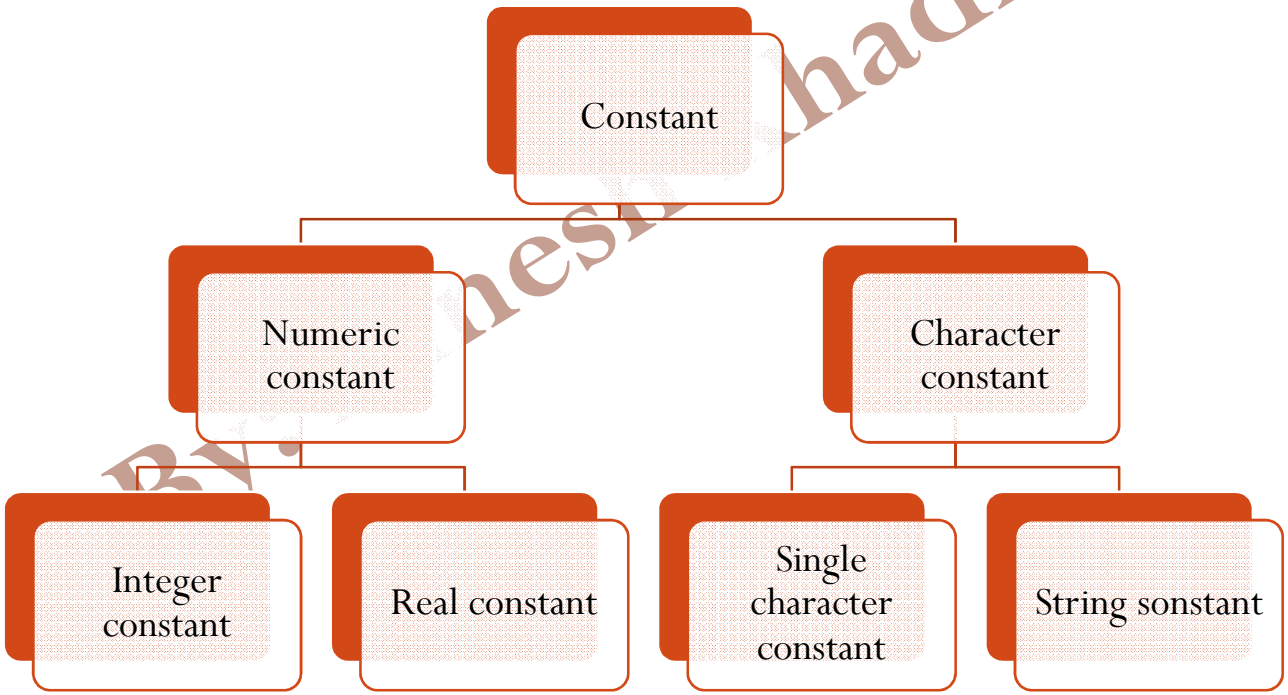
  enum day {Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday};

  enum values {True, False};

By: Dinesh Khadka

Constants

- A constant is an entity that doesn't change whereas a variable is an entity that may change.
- A constant is an entity that doesn't change whereas a variable is an entity that may change.
- Constants are classified as



By: Dinesh Khadka

Integer constants

- An integer constant refers to sequence of digits. There are three types of integers – decimal integer, octal integer, and hexadecimal integer.
- Decimal integers consists of a set of digits, 0 to 9, preceded by – or + sign. Examples of valid integer constants are: 123 -234 0 +78
- Octal integer constant consists of any combination of digits from set 0 through 7, with leading 0. Some examples of octal integer constants are: 023 0435 0553
- A sequence of digits preceded by 0x or 0X is hexadecimal integer constant. They may also include alphabets A to F. The letters A to F represent numbers 10 to 15. Following are the example of valid hex integers: 0X2 0x7F 0xBBA
- **Rules for Constructing Integer Constants**
  - An integer constant must have at least one digit.
  - It must not have a decimal point.
  - It can be either positive or negative.
  - If no sign precedes an integer constant it is assumed to be positive.
  - No commas or blanks are allowed within an integer constant.
  - The allowable range for integer constants is -32768 to 32767 in 16 bit system. (or system dependent).
- Note: we rarely used octal and hexadecimal number in programming.

By: Dinesh Khadka

Real constants

- Integer numbers are not sufficient to represent quantities that vary continuously, such as distances, heights, temperatures, price and so on.
- These quantities are represented by numbers containing fractional parts like 12.456 such numbers are called real (or floating point) constants. E.g. are: 0.0089
  -0.54 345.54 +43.0
- The real constant can be written in two forms – fractional forms and exponential forms.
- **Following rules must be observed while constructing real constants expressed in fractional form:**
  - A real constant must have at least one digit.
  - It must have a decimal point.
  - It could be either positive or negative.
  - Default sign is positive.
  - No commas or blanks are allowed within a real constant.
  - Ex.: +325.34 ,426.0 , 32.76 , -48.5792

By: Dinesh Khadka

- The exponential form of representation of real constants is usually used if the value of the constant is either too small or too large.
- In exponential form of representation, the real constant is represented in two parts. The part appearing before 'e' is called mantissa, whereas the part following 'e' is called exponent.
- **Following rules must be observed while constructing real constants expressed in exponential form:**
  - The mantissa part and the exponential part should be separated by a letter e.
  - The mantissa part may have a positive or negative sign.
  - Default sign of mantissa part is positive.
  - The exponent must have at least one digit, which must be a positive or negative integer. Default sign is positive.
  - Range of real constants expressed in exponential form is -3.4e38 to 3.4e38.
  - Ex.: +3.2e-5 ,4.1e8, -0.2e+3, -3.2e-5

By: Dinesh Khadka

Single character constant

- A single character constant (or simply character constant) contains a single character enclosed with in a pair of single quote marks.
- E.g. are: 'X', '5', ';', ' ' etc.

String Constants

- A string constant is a sequence of characters enclosed in *double* quotes.
- E.g. "Hello", "1987", "well done", "x" etc

- Note: remember that a character constant (e.g. 'x') is not equivalent to the string constant (e.g. "x").

By: Dinesh Khadka

Variable

- A variable is a data name that may used to store a data value.
- Variables are entities whose value may change through out the program execution.

Declaration of variables

- Variable should be declared before its use.
- After designing the suitable variables names, we must declare them to the compiler. Declaration does the following two things.
  - It tells the compiler what the variable name is.
  - It specifies what type of data the variable will hold.
- **Syntax for variable declaration is :**

  data-type v1, v2, …, vn;

  where v1, v2 ,…., vn are the variable names. Variables are separated by commas. And declaration statement must end with semicolon.
- Examples

```
int a, b, sum;      //a , b and sum are integer variables
float area;         // area is float variable
char ch;            // ch is char variables
double x,y;         //x and y are double variables
```

By: Dinesh Khadka

Variable[cont…]

Rules for constructing variables name

- A variable name consists of letters, digits, underscore (_) character, subject to the following conditions:
  - They must begin with a letter. Some system permits underscore as the first character.
  - Length should not normally more than 8 character, but ANSI permit a length of 31 characters.
  - Upper and lower case are significant. Hence the variable **Total** is not same as **total** or **TOTAL.**
  - It should not be keyword.
  - White space is not allowed.
- Some example of valid variables name are:

  distance , sum1, ph_value, _x1, aetc
- Some example of invalid variables name are:

  123, (area), %,25th , group one, auto etc.

By: Dinesh Khadka

```
/*program that uses variables
 for adding two numbers*/

#include<stdio.h>
#include<conio.h>

void main()
{
    int n1,n2,sum;
    printf("Enter first number: ");
    scanf("%d",&n1);
    printf("Enter Second number: ");
    scanf("%d",&n2);
    sum=n1+n2;
    printf("%d +%d = %d",n1,n2,sum);
    getch();
}
```

By: Dinesh Khadka

Escape sequences

- Non-printable characters are called escape sequences.
- It is a character combination consisting of a backslash (\) followed by a letter or a digit.
- Escape sequences always represent single characters, even though they are written in terms of two or more characters.
- Escape sequences has a single ASCII value.
- Escape sequences are useful for formatting input and output.
- It is called escape sequences because it cause an 'escape' from normal way the character are interpreted.
- some of the common escape sequence are:

Escape sequence	Meaning	Escape sequence	Meaning
\a	Bell/ Beep	\b	Backspace
\n	New line	\r	Carriage return
\t	Horizontal tab	\\	Backslash
\'	Single quote	\"	Double quote
\v	Vertical tab	\?	Question mark
\0	Null character		

Dinesh Khadka

Example

```
/* program to demonstrate
use of escape sequence*/

#include<stdio.h>
#include<conio.h>

void main()
{
   printf("Hello \"Class\"");
   printf("\nWelcome to C programming");
   getch();
}
```

By: Dinesh Khadka

Symbolic constant

- A symbolic constant is a name that substitutes for a sequence of characters.
- The character may represent a numeric constant, a character constant or a string constant.
- When in a program is compiled, each occurrence of symbolic constant is replaced by corresponding constant.
- Symbolic constant are defined at the definition section of the program.
- General form of symbolic constant definition is:

#define name constant

Here

- name -> represent symbolic name normally written in upper case.
- constant -> sequences of characters associated with symbolic name.

By: Dinesh Khadka

Symbolic constant cont…

Examples:

```
#define   PI      3.141593
#define  TAXRATE        0.15
#define TRUE    1
#define  MESSAGE  "Hello class"
```

Rules

- Symbolic constant names are same as variable names. Convention: Use upper case letters while defining symbolic constants.
- No blank space permitted between # and define.
- A blank space is required between #define and symbolic name and between symbolic name and its value (i.e. constant).
- No = sign between name and constant
- No semicolon(;) at the end

Assignment: WAP to find area and perimeter of circle using symbolic constant.

By: Dinesh Khadka